Popravni test iz predmeta
Algoritama i struktura podataka

Drugi praktični test održan 13.01.2024. Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
    a. Vodite računa da imate sve spremno što vam je neophodno.
    b. Isključite mobilne telefone i odložite ih na katedru.
4. 0Ukupan broj bodova na testu je 30 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na nazbačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP_Test2_GrupaJ_IT1g2021**
8. Ne postoji minimum bodova koji se mogu osvojiti iz svakog pojedinačnog zadatka
9. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova**
10. Ukoliko je u napomeni za rešavanja navedeno da je potrebno **koristiti metode date u šablonu**, da bi se ostvario maksimalan broj bodova **neophodno je njihovo korišćenje**. Tačno rešenje koje ne koristi ove algoritme se boduje brojem bodova manjim od maksimalnog.
11. Ukoliko za algoritam (rešenje zadatka) nije obezbeđen poziv odgovarajuće metode u *Main*-u kojom se algoritam testira, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova**

Pre početka testa napišite ime i prezima i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Implementacija DFSVisit metode Depth-First-Search algoritma*

Napisati implementaciju DFSVisit metode istoimenog algoritma.

a) Napisati algoritam.
b) U *Main* metodi kreirati proizvoljan graf. Obezbediti način za testiranje i proveru nove funkcionalnosti, odnosno poziv ***DFS*** metode.

Dodatne napomene za rešavanje zadatka:

- U template-u je metoda ***private void DFSVisit(LinkedList.Node node, int nodeIndex)*** koja je osnovna metoda za rekurzivno izvršavanje DFS algoritma.

Zadatak 2 – *Ispisivanje svih karaktera koji se ponavljaju više od 2 puta*

Napisati algoritam vremenske kompleksnosti O(n) koji će iz string-a koji se prosleđuje identifikovati karaktere koji se ponavljaju više od 2 puta i ispisati ih u konzoli.

Dodatne napomene za rešavanje zadatka:

- Koristiti *Hash* tabelu koja je data u šablonu.
- Implementaciju pisati u okviru zakomentarisane metode ***public static void printAllRepeatings(string str).***
- Algoritam realizovati tako da **ne postoji** razlika između malih i velikih karaktera.
- Kako biste string konvertovali u niz karaktera sa kojim se može vršiti manipulacija, koristiti metodu ***ToCharArray()*** po principu: ***str.ToCharArray().***
- Karakteri se ne moraju eksplicitno konvertovati u celobrojni tip prilikom ubacivanja u *Hash* tabelu**.**
- Redosled karaktera prilikom ispisivanja nije bitan.

Primer:

Input 1:	*"Algoritam"*
Output 1:	*""*
	(Ne postoji karakter koji se ponavlja više od 2 puta)
Input 2:	*"Strukture podataka"*
Output 2:	*"ta"*
	(Karakteri „a" i „t" se pojavljuju po 3 puta)

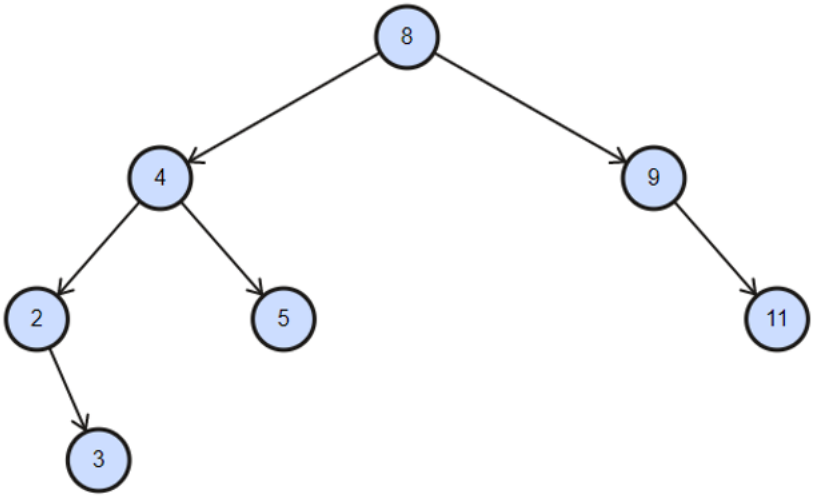
Zadatak 3 – Pronalaženje podstabala sa neparnim brojem parnih čvorova

U okviru standardne strukture klase ***BinarySearchTree*** napisati implementaciju metode koja će omogućiti ispis korena podstabala koja imaju neparan broj čvorova sa parnom vrednošću.

Dodatne napomene za rešavanje zadatka:

- U okviru klase ***BinarySearchTree*** kreirati javnu metodu sa potpisom ***public void findOddEvensSubtree()***.
- Iz prethodno kreirane javne metode treba da se poziva privatna metoda sa potpisom ***private int findOddEvensSubtree(Node curr)***
- Privatna metoda treba da sadrži implementaciju algoritma kojim će se prebrojavati čvorovi sa parnom vrednošću i za svako podstablo proveravati da li je broj čvorova sa parnom vrednošću neparan. U slučaju da jeste, potrebno je u konzoli ispisati vrednost korena ovog podstabla.
- Listove ne treba posmatrati.
- Algoritam **OBAVEZNO** pisati u okviru klase ***BinarySearchTree***

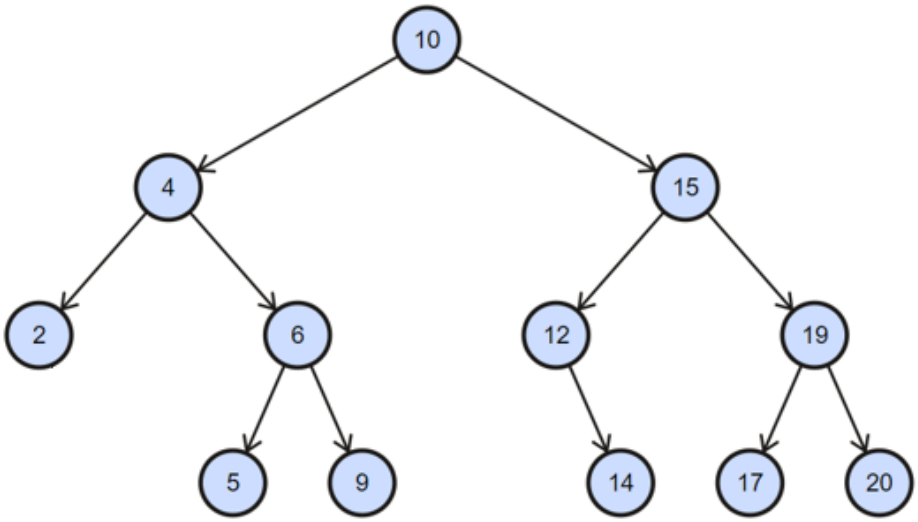
Primeri:



Slika 1 - Primer BST-a

Input 1: *BST sa slike 1*

Output 1: *4*



Slika 2 - Primer BST-a

Input 2: *BST sa slike 2*

Output 2: *15, 12, 19*